

Distributed Systems - Notes - v0.1.0

260236

September 2026

Preface

Every theory section in these notes has been taken from the sources:

- Course slides. [1]

About:

 [GitHub repository](#)



These notes are an unofficial resource and shouldn't replace the course material or any other book on distributed systems. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

Contents

1	Introduction	4
1.1	Why Distributed Systems?	4
1.1.1	Motivation: Why Distributed Systems?	4
	Index	14

1 Introduction

1.1 Why Distributed Systems?

1.1.1 Motivation: Why Distributed Systems?

There is a very simple reason why distributed systems are important: most applications we use today are **distributed**. Many applications cannot realistically run as a single program on a single computer.

Example 1: Distributed Applications

E-mail, web/cloud applications such as *Google Maps*, *Google Docs*, *Microsoft Office 365*; mobile applications and mobile devices, web services and websites, and many more, are all distributed systems.

E-mail is used as an especially clear example because sending a message involves several independent machines communicating over a network:

User A → mail server A → network → mail server B → User B

There is no single computer performing the entire operation.

So the first intuition is that **modern applications are naturally composed of multiple communicating machines**. This is why distributed systems are not some niche branch of computer science, they are behind a huge fraction of modern computing infrastructure.

State of the Internet

Censys on 2023 created a report about the **observable public Internet**, built from large-scale Internet scanning.¹ The report says that in one daily snapshot in early 2023, Censys observed more than **740 million hosts** exposing about **1.3 billion HTTP services**, roughly **18%** hosted by major cloud providers such as Amazon, Oracle, Google Cloud, or Azure. And more than **85%** of HTTP services running on named hosts.

 A distributed system may therefore consist not of two or three machines, but potentially of 10^3 , 10^5 , 10^6 , or even more interacting components. But this immediately hints at problems that will appear later in the notes: scalability, failures, communication, naming, synchronization, replication, and so on. But at this point, we are just trying to understand why distributed systems are important.

Large-scale Web traffic

From the same Censys report, we can also see the **traffic** perspective rather than the number of machines. For example, the report says that in 2023, on Google:

¹Censys is a platform that **scans and indexes Internet-connected systems and services**. It is like a search engine for Internet infrastructure.

- More than **100k searches per second**
- More than **100 billion visits per month**
- YouTube as another extremely large service
- Google and YouTube producing more traffic than many other major websites combined

Again, the exact values are secondary. The important question is: *could a single server handle all of this?* Clearly not. A large Web service therefore has to distribute work across many machines. And this is the first glimpse of the **scalability** problem that distributed systems have to solve.

In summary, **modern computing operates at a scale where distribution is not just a design choice, but unavoidable.** We saw how widespread distributed applications are, the size of the Internet, and finally the traffic generated by huge Web services.

🌲 LLM infrastructures as distributed systems

Let's consider what happens when we send a query to a modern Large Language Model (LLM) such as ChatGPT. It is tempting to imagine:

Our request → one very powerful GPU → our answer

But this is not how it works. Modern LLMs are too large to fit on a single GPU. So a single query to a modern LLM may involve roughly **16 to 32 GPUs**, corresponding to a couple of multi-GPU nodes. Each GPU may consume up to about **700 W**.

So even a single logical request may require computation spread across several physical processors:

$$\text{Request} \rightarrow \begin{cases} \text{GPU}_1 \\ \text{GPU}_2 \\ \text{GPU}_3 \\ \vdots \\ \text{GPU}_{16-32} \end{cases} \rightarrow \text{Answer}$$

From the user's perspective, however, we do not see any of this. We simply **interact with one service: LLM service.** This is already one of the central ideas of distributed systems: **many independent computational resources cooperate to provide what appears to the user as one single service.** The infrastructure becomes even more clearly distributed when we move from one request to the whole data center. An AI data center may contain **more than 100,000 AI accelerators**, with power consumption potentially reaching hundreds of megawatts.

🧩 So there are actually **two different forms of distribution** here:

- At the level of a **single request**:

One Computation $\rightarrow$ Multiple GPUs

- At the level of the **service as a whole**:

Millions of requests $\rightarrow$ Many machines $\rightarrow$ Many clusters
 $\rightarrow$ Possibly many data centers

The reason this example matters is that distribution is not only about connecting independent applications. Sometimes even **one computation is too large to execute efficiently on one processor**. That naturally connects distributed systems with high-performance computing (HPC), although the two are not identical. However, **modern AI services are distributed both because of computational size and service scale**.

🧩 Google as a distributed “supercomputer”

Eric Schmidt (CEO of Google from 2001 to 2011) described Google as essentially “*a large distributed supercomputer spread across many data centers*”. That is a very useful mental model. Instead of imaging Google as a single website, we can think of it as an **enormous collection of cooperating computers** spread geographically across the world.

Data center A $\leftrightarrow$ Data center B $\leftrightarrow$ Data center C $\leftrightarrow$...

And inside each data center, there are many cooperating machines:

server₁, server₂, server₃, ...

All connected through networking infrastructure.

Google historically operated hundreds of thousands of servers and gives a more recent estimate of **more than 2 million servers across 36 data centers**. The important point is not the exact count; it is the architectural idea. From outside, we might type `google.com` and perceive a single service. But internally, that service may involve an enormous distributed infrastructure. So we get an important contrast:

- **User’s view**: Google, a single website.
- **Physical reality**: millions of machines, networks, and data centers cooperating to provide a single service.

This is a recurring objective in distributed systems: **expose a coherent service while hiding much of the underlying distribution**. We will later give this idea a precise name: **transparency**.

⚠️ There is also another important implication. If the system contains millions of machines, failures cannot be treated as exceptional events. Suppose each individual machine is highly reliable. With millions of components, some machine

somewhere will still fail regularly. So at very large scale, **failure** is not something that might happen once in a while. It **becomes something the system must be designed to live with**. This point will become central when we discuss **partial failures** and **fault tolerance** later. For now, the main takeaway is that **a huge distributed infrastructure can behave externally like one computer/service**.

☰ Distributed systems beyond the Internet

At this point it would be easy to conclude that distributed systems are only internet applications. That would be **wrong**. The Internet, the Web, and LLM platforms are distributed systems, but many distributed systems have little or nothing to do with the public Internet.

Example 2: Distributed Systems Beyond the Internet

- **Intranets**. An organization can have a private distributed system. For example, banking systems, corporate systems, scientific computing infrastructure, server farms, grids, clouds.
For example, imagine a bank with branches in Milan, Rome, and Turin. The system is distributed even if its internal communication is largely private and not exposed to the public Internet.
- **Home networks**. Even our home can contain a small distributed environment: laptop, router, tv, NAS, phone, and so on. Different devices communicate and share resources. This is a distributed system on a dramatically smaller scale than Google, but the idea is the same: independent devices and communication.
- **Personal Area Network (PAN)**. A person's devices can themselves form a small network: phone, smartwatch, headphones, gps. For example healthcare-monitoring devices can communicate with a phone and send data to a cloud service. Again, **distribution does not imply thousands of servers. Two or three cooperating devices are already enough to introduce distributed-system concepts**.
- **Wireless Sensor Networks**. Another important category consists of many small sensing devices communicating with one another or with gateways. For example, a smart home may have many sensors and actuators communicating with a central hub. Or a smart city may have thousands of sensors monitoring traffic, pollution, or infrastructure health. These are distributed systems even if they are not part of the public Internet.

Thus a distributed system may exist across continents, inside one company, inside one building, inside one house, or even around one person. What matters is not the geographical size, but that **multiple independent computational components communicate and cooperate**.

🏠 Home and personal networks

Until now, we have mostly thought about distribution at a very large scale: Google, cloud infrastructures, LLM data centers. But a distributed system can also exist on a much smaller scale.

For example, inside a home we might conceptually have a PC, a laptop, a media device, connected through a local network and possibly communicating with remote machines through the Internet. So **distribution is not defined by physical distance**. Two computers sitting in the same room can form part of a distributed system just as computers located on different continents can.

In the previous page, we mentioned home networks and Personal Area Networks (PANs). A home network concerns devices belonging to the environment:

$$\text{home} \rightarrow \{\text{computers, entertainment devices, shared resources}\}$$

A personal-area network concerns devices surrounding a person:

$$\text{person} \rightarrow \{\text{phone, smartwatch, headphones, gps, etc.}\}$$

⚠️ The scale is small, but the distributed-system issue remains: **independent components must communicate and coordinate**.

📡 Pervasiveness of Distributed Systems

Instead of having a few clearly recognizable computers (desktop, laptop, server), **computing and communication capabilities become embedded in ordinary physical objects**. This is the basic idea behind the **Internet of Things (IoT)**. We can use three quotations to develop this idea:

1. **Computing that “disappears”**. Mark Weiser (1991) says that the most profound **technologies** eventually become **integrated into everyday life** to the point that we **stop consciously noticing them**. This is the idea of **Ubiquitous Computing** (or **Pervasive Computing**).
Think about the conceptual transition from “*use a computer*” toward “*computing is embedded in the environment around us*”. The computer itself is no longer necessarily the object we interact with.
2. **Connectivity for things**. The ITU (International Telecommunication Union) quotation captures another transition. Traditional networking gave us connectivity from anyone to anywhere (i.e., from one person to another). IoT extends this toward **connectivity for anything**. Objects that previously had no communication capabilities can contain small transceivers and participate in a network. So instead of only connecting computers, we may have a network of connected objects. The number of communicating entities consequently becomes enormous, **21.1 billion active IoT connections by the end of 2025**.
3. **From connected objects to intelligent objects**. The Negroponte quotation adds another important point. IoT is not merely attaching an

identifier to an object, such as an RFID tag that tells us where the object is. IoT is about **embedding intelligence into objects**. So a device can potentially:

- (a) Observe something about its environment (e.g., temperature, humidity, motion, light, etc.)
- (b) Process information (e.g., detect patterns, make decisions, etc.)
- (c) Communicate with other devices (e.g., send data, receive commands, etc.)
- (d) Participate in a larger system (e.g., a smart home, a smart city, etc.)

This gives us a useful progression: from ordinary objects, to connected objects, to **networked computational components**. And once we have potentially billions of these components, distributed-system techniques become essential.

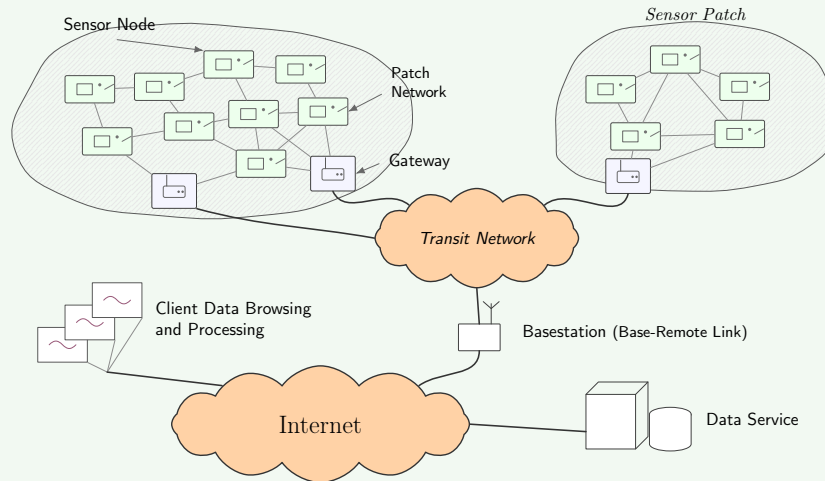
Example 3: Why a Wireless Sensor Network is a pervasive system

Imagine we want to monitor a foster, a factory, a building, or a crop field. Instead of putting one big computer in the middle, we scatter many tiny devices around the environment:

$$S_1, S_2, S_3, \dots, S_n$$

Each one may measure something like: temperature, humidity, light, vibration, pressure, motion, etc. These devices are the **sensor nodes** and they are placed on the edge of the environment.

The crucial idea is that computing becomes part of the environment itself. We are no longer just using a computer to monitor the environment. Instead, the sensors are just “there”, continuously observing the physical world. That is exactly why this is **pervasive computing**: computation is distributed throughout the environment, and it is not necessarily associated with a single device that we consciously interact with.



In the previous diagram, we represent a generic architecture for a Wireless Sensor Network (WSN).

- On the top-left, we have small devices inside the shaded region, called **sensor nodes**. Each one is a small computer with, conceptually, a CPU, memory, and a small radio transceiver. The sensor observes something physical, for example temperature, humidity, light, vibration, pressure, motion, etc.
- A group of sensor nodes may form a **Sensor Patch** (top-right), which is a small area of the environment that is monitored by a group of sensors. Inside each patch, the sensor nodes communicate through the **Patch Network**. Therefore, the sensors form a local network rather than each sensor independently connecting directly to the Internet.
- At the edge of each Sensor Patch, we have a **Gateway**. The sensor network may use some specialized local wireless technology, while the rest of the system uses a different network. The gateway bridges the two networks. **So instead of each sensor node connecting directly to the Internet, the gateway collects data from the sensor nodes and sends it to the Internet.**
- Multiple gateways connect to the **Transit Network**. This part simply transports information from the local sensor patches toward a more conventional infrastructure.
- The transit network reaches a **Base Station**. This is the point where data can be forwarded toward a larger network infrastructure. We call it also a **Base-Remote Link**, because it connects the local sensor network to a remote infrastructure.
- Once the information reaches the Internet, it can be sent to the **Data Service** shown on the bottom-right. This might conceptually be a database, a server, a cloud service, or an analytic system.

For example, suppose the sensors are measuring temperature. The data service could receive: $(t_1, 22.4^\circ C)$, $(t_2, 22.5^\circ C)$, $(t_3, 22.6^\circ C)$, and store or process those values.

- Finally, on the left of the Internet cloud, we have **Client Data Browsing and Processing**. That is simply the user/application consuming the collected data.

For example, a control software could receive the temperature data and decide to turn on a fan or an air conditioner if the temperature exceeds a certain threshold.

This example shows that distributed systems are not just servers communicating over the Internet. They can also be pervasive systems, where computation is embedded in the environment itself.

❓ Why mobility makes distributed systems harder

We can consider three different mobility scenarios that make distributed systems more challenging:

- **Normadic**. In the nomadic case, the device moves between locations, but communication is associated with whichever fixed network access point it is currently using.

For example, a laptop may move between different Wi-Fi access points in a building. Thus, the **device changes its point of attachment** to the network, and the natural question is “*how do we continue referring to the same device when its network location changes?*”. This question will become important later when we discuss *naming*.

- **Base-station mobility**. Think about a mobile phone on a cellular network. We are moving, but there is a fixed infrastructure around us. Suppose we are walking through Milan while on a call. At first our phone communicates through BS_1 , then we move far enough that another base station becomes better, so we switch to BS_2 .

The important thing is that the **device moves, but the infrastructure does not**. The network has fixed access points/base stations, and the mobile node attaches to whichever one is appropriate. The distributed-system challenge is that the device’s **point of attachment changes while the device itself remains the same entity**. So again, identity and current location must be kept separate.

- **Ad-hoc mobility**. Imagine several laptops or phones in the same area, and they can communicate directly with one another through their wireless interfaces.

$$A \leftrightarrow B \leftrightarrow C$$

If A cannot directly reach C , but B can reach both, then B may conceptually act as an intermediate node. The important feature is that **the devices themselves form the network**. There is no fixed base station organizing the connectivity.

For example, imagine several rescue teams entering an area where there is no working network infrastructure. Their devices may form a temporary network among themselves:

$$A \leftrightarrow B \leftrightarrow C \leftrightarrow D$$

Then people move. Maybe B leaves the area, so the topology changes:

$$A \leftrightarrow C \leftrightarrow D$$

Or perhaps A can no longer reach the others at all. That is why ad-hoc networks are especially dynamic: the **network topology itself changes as the nodes move**.

The three cases can be summarized as follows:

Model	Device moves?	Fixed infrastructure?
Nomadic	Yes, between attachment points.	Yes
Base-Station	Yes	Yes, explicitly through base stations.
Ad-Hoc	Yes (potentially)	No, fixed base station.

Even without mobility, a distributed system already needs to know:

- *Who is the other component?* (identity)
- *How can I reach it?* (location)

With mobility, the second answer may continuously change.

References

- [1] Margara Alessandro Cugola Gianpaolo Saverio. Distributed systems. Slides from the HPC-E master's degree course on Politecnico di Milano, 2026.

Index

P

Pervasive Computing 8

U

Ubiquitous Computing 8